

Blokus - Rapport IA

Groupe :

- Robin Ferreira
- Khoodaruth Tatiana
- Soidrine Saindou
- Calvin Pichon
- Omar Barkok
- Marouane Ait-Hanine

Date de soutenance :

Lundi 2 Juin 2025

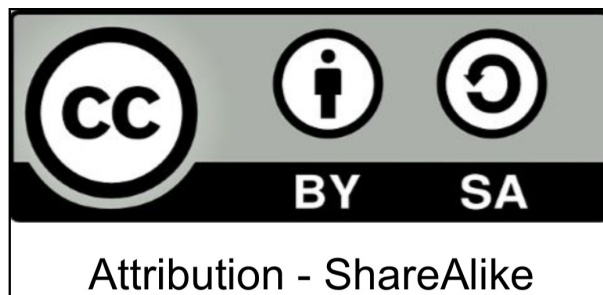


Table des matières

1	Introduction	2
2	Description des algorithmes	2
2.1	IA facile	2
2.2	IA intermédiaire	2
2.2.1	MiniMax Alpha-Beta Pruning	2
2.2.2	Heuristique	4
2.2.3	Idées de Stratégies	5
2.3	IA difficile - Idée d'implémentation	5
3	Synthèse des évaluations effectuées	6
3.1	Niveaux de difficulté distincts	6
3.2	Jeu contre des humains	6

1 Introduction

D'après notre planning et nos ambitions, nous avons l'intention de faire une IA facile, intermédiaire et difficile. En raison de difficulté au niveau de l'intégration tout au long du projet, nos IA n'ont pu être testées de façon optimale.

2 Description des algorithmes

2.1 IA facile

Pour la conception du niveau le plus bas, nous avons développé une IA aléatoire qui, à partir d'une liste de coups possibles, en choisit un au hasard puis le joue. Nous avons envisagé qu'il serait préférable de concevoir une IA qui, au lieu de jouer aléatoirement, sélectionnerait l'un des moins bons coups afin de favoriser un joueur débutant n'ayant pas encore de stratégie propre. Cependant, cette idée a finalement été abandonnée, et nous avons simplement conservé la version aléatoire.

Data: Liste des coups possibles L

Result: Un coup choisi aléatoirement parmi L

$r \leftarrow$ entier aléatoire entre 0 et $|L| - 1$;

$coup \leftarrow L[r]$;

return $coup$;

Algorithm 1: IA_aléatoire

2.2 IA intermédiaire

2.2.1 MiniMax Alpha-Beta Pruning

Concernant l'IA de niveau intermédiaire, nous nous sommes penchés sur un algorithme MiniMax avec élagage alpha-bêta.

La classe se compose d'un constructeur et de deux méthodes : `minimaxAB` et `execution`.

La première méthode (*minimaxAB*) construit un arbre de valeurs, où la racine contient une instance du plateau du jeu et chaque nœud fils contient un coup spécifique joué. La seconde méthode (*execution*) appelle la première, récupère la meilleure valeur, puis la renvoie.

Data: Coup initial, tableau de joueurs J , index courant i , profondeur d , bornes α, β

Result: Un entier représentant le meilleur score trouvé

```

 $j \leftarrow J[i];$ 
 $L \leftarrow$  liste des coups jouables pour  $j$ ;
if  $d = 0$  ou  $L$  est vide then
    | return valeur de la fonction d'évaluation;
end
if  $j$  est l'IA then
    |  $meilleur \leftarrow -\infty$ ;
end
else
    |  $meilleur \leftarrow +\infty$ ;
end
foreach  $coup \in L$  do
    | jouer( $coup$ );
    |  $score \leftarrow$  MiniMaxAB( $coup, J, (i + 1)|J|, d - 1, \alpha, \beta$ );
    | annuler( $coup$ );
    | if  $j$  est l'IA then
    | |  $meilleur \leftarrow \max(meilleur, score)$ ;
    | |  $\alpha \leftarrow \max(\alpha, score)$ ;
    | end
    | else
    | |  $meilleur \leftarrow \min(meilleur, score)$ ;
    | |  $\beta \leftarrow \min(\beta, score)$ ;
    | end
    | if  $\alpha \geq \beta$  then
    | | break ;
    | end
end
return  $meilleur$ ;

```

// Élagage alpha-bêta

Algorithm 2: MiniMaxAB

Pour la méthode execution, on suit pratiquement le même schéma, à la différence qu'elle renvoie le meilleur coup et non la meilleure valeur. On initialise un entier maxValeur, puis, dans une boucle semblable à celle décrite précédemment, on procède ainsi :

Data: Plateau actuel, joueurs J , index courant i , profondeur d , bornes α , β
Result: Le meilleur coup à jouer pour l'IA
 $maxValeur \leftarrow -\infty$;
 $meilleurCoup \leftarrow null$;
 $L \leftarrow$ liste des coups jouables pour $J[i]$;
foreach $coup \in L$ **do**
 jouer($coup$);
 $score \leftarrow$ MiniMaxAB($coup$, J , $(i + 1)|J|$, $d - 1$, α , β);
 annuler($coup$);
 if $score > maxValeur$ **then**
 $maxValeur \leftarrow score$;
 $meilleurCoup \leftarrow coup$;
 end
end
return $meilleurCoup$;

Algorithm 3: execution

À l'origine, le coup ne faisait pas partie des paramètres du Minimax, mais nous l'avons ajouté afin de pouvoir l'utiliser dans l'heuristique et ainsi améliorer la précision de l'évaluation.

2.2.2 Heuristique

Pour les heuristiques, nous avons commencé assez simplement, avec une évaluation basée uniquement sur le score du joueur courant, ce qui pousse l'IA à privilégier les coups jouant les pièces les plus grandes. Dans le même esprit, nous avons développé d'autres fonctions : l'une renvoie le nombre de pièces encore disponibles, une autre le nombre de coins accessibles pour les adversaires, et une autre encore évalue le nombre de coups possibles pour les adversaires.

Par la suite, nous avons combiné plusieurs de ces fonctions simples afin d'observer les comportements que l'IA pouvait adopter. Des pondérations sous forme de bonus ou de pénalités sont appliquées à chaque fonction, et leurs résultats sont multipliés par ces coefficients pour ajuster l'évaluation globale.

Pour le Minimax, nous avons finalement retenu une heuristique dynamique qui varie en fonction de l'avancement de la partie : elle adopte un comportement spécifique en début, milieu et fin de partie.

- **En début de partie**, on accorde davantage de poids au score de l'IA, auquel on soustrait la distance de la pièce jouée au centre du plateau. L'objectif est d'inciter l'IA à se diriger rapidement vers le centre avec de grosses pièces, plus difficiles à placer en fin de partie. (arbre de profondeur 1)
- **En milieu de partie**, le score est toujours pris en compte, mais avec une pondération plus faible. On y ajoute le nombre de coins disponibles multiplié par un bonus de coin, afin de favoriser les coups qui offrent plus d'options. Ensuite, on soustrait le nombre de coins disponibles pour les adversaires, multiplié par une

pénalité. (arbre profondeur 1 car une profondeur plus grande rend le jeu injouable)

- **En fin de partie**, la combinaison reste la même, mais les pondérations sont ajustées pour s'adapter à cette phase spécifique du jeu. (arbre de profondeur 4 quand il reste moins de 10 coups jouables)

2.2.3 Idées de Stratégies

En jouant plusieurs parties, nous avons rapidement compris qu'étaler ses pièces efficacement sur le plateau était essentiel, tout comme le fait de bloquer les possibilités de l'adversaire. Malheureusement, nous n'avons pas réussi à intégrer concrètement une logique de blocage ciblé, même si l'une des idées envisagées était d'isoler un adversaire dans un coin afin de limiter au maximum ses possibilités de déplacement.

Parmi les autres idées que nous n'avons pas pu implémenter, on peut citer :

- conserver certaines pièces spécifiques dans l'optique de bloquer un adversaire.
- se diriger vers le coin opposé au point de départ.
- isoler un joueur adverse.
- se réserver une zone pour y placer une pièce en fin de partie, lorsque les options centrales sont épuisées.
- conserver certaines pièces spécifiques dans l'optique de bloquer un adversaire.

Étant donné que le parcours en profondeur 2 devient déjà assez long en milieu de partie, une alternative possible serait de simuler uniquement certains coups identifiés comme dangereux ou stratégiquement forts, afin de mieux y réagir sans avoir à parcourir tous les coups à chaque fois. De la même manière, au début de la partie, il n'est pas forcément nécessaire d'explorer toutes les possibilités, même en profondeur 1, puisque l'on sait qu'il est optimal d'utiliser des pièces de 5 cases. Il serait alors plus judicieux de définir une liste prédéfinie de ces pièces, afin d'éviter des calculs inutiles.

Bref, en appliquant ces optimisations, on pourrait garantir à la fois une certaine fluidité dans l'exécution de l'IA et un niveau d'adversité intéressant, notamment en début et milieu de partie. Cela permettrait ensuite d'augmenter la profondeur de recherche à 2, voire 4, en fin de partie, où le nombre de coups disponibles est bien plus limité.

2.3 IA difficile - Idée d'implémentation

Dans le but de concevoir une IA plus difficile à battre, nous avons réfléchi à une amélioration de notre algorithme MinMax avec élagage alpha-bêta, en jouant à la fois sur la profondeur adaptative de l'arbre de recherche et sur des pondérations dynamiques des heuristiques.

L'idée était de faire évoluer le comportement de l'IA en fonction de l'avancement de la

partie, pour la rendre plus stratégique et plus réaliste. Cela aurait permis à l'IA d'ajuster ses priorités en temps réel, en s'appuyant sur une combinaison pondérée de différentes fonctions d'évaluation : score, mobilité, coins accessibles, dangerosité des coups adverses, etc.

Malheureusement, malgré l'intérêt de cette idée, nous n'avons pas eu le temps d'implémenter cette version plus avancée de l'IA. Le développement s'est concentré sur une version fonctionnelle et stable basée sur des heuristiques fixes. Ce travail reste cependant une piste d'amélioration évidente et prometteuse pour des versions futures de l'IA.

3 Synthèse des évaluations effectuées

3.1 Niveaux de difficulté distincts

Lors de cette série de 50 parties opposant une IA utilisant l'algorithme Min-Max à trois IA fonctionnant de manière aléatoire, les résultats montrent une nette supériorité de la stratégie Min-Max. L'IA Min-Max a remporté 47 des 50 parties, n'en perdant que 3 : deux fois en terminant en deuxième position (parties 7 et 31) et une fois en troisième position avec un score très serré (partie 41). Elle a également terminé ex æquo à deux reprises (parties 24 et 50). Ces performances soulignent l'efficacité de l'approche Min-Max, qui parvient non seulement à remporter la grande majorité des parties, mais aussi à maintenir une régularité remarquable face à plusieurs adversaires jouant de manière aléatoire.

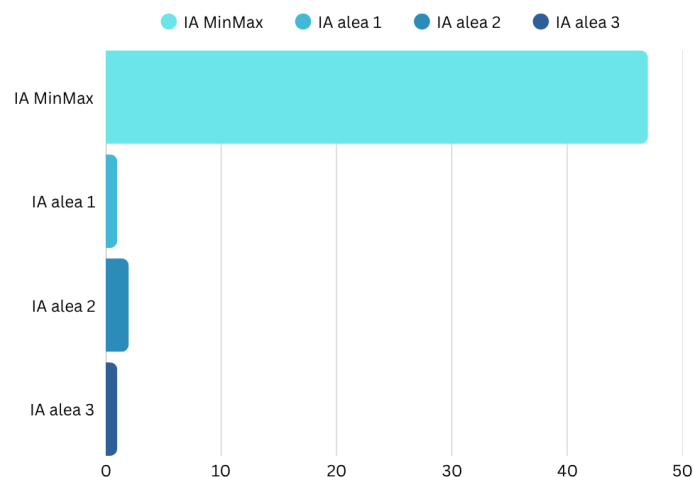


FIGURE 1 – Nombre victoires IA MinMax vs 3 IA aléatoire

3.2 Jeu contre des humains

Humain débutant vs IA MinMax L'IA MinMax a de fortes chances de l'emporter face à un joueur débutant. Elle privilégie l'utilisation des pièces les plus grandes en début

de partie, tout en se dirigeant stratégiquement vers le centre du plateau. Cette approche lui permet de maximiser son expansion et de mieux bloquer ses adversaires. D'autant plus que par la suite elle cherche à minimiser les coins disponibles de son adversaire pour lui laisser moins de choix, ce qu'un joueur débutant ne maîtrise généralement pas encore.

Humain avancé vs IA MinMax Dans ce cas, l'avantage peut basculer en faveur du joueur humain. Un joueur ayant acquis quelques notions de stratégie connaît certaines méthodes efficaces pour contrer les choix typiques de l'IA. Par exemple, il saura éviter de s'exposer à des blocages ou cherchera à limiter l'expansion de l'IA en l'encerclant progressivement.